

Fiche de révision : Oral E6 · ColorRoom · Téo Tromprier (E2)

Format : 20 min présentation (PDF) → 20 min démonstration (app) → 20 min questions.

Objectif : connaître le projet ET le code. Tout ci-dessous est tiré du dépôt réel

NewGenesisAgency/color_room.

1. Pitch (30 secondes, à savoir par cœur)

La **ColorRoom** est un équipement scientifique du laboratoire **ENTPE / LTDS / BPMNP**, hébergé à **LUMEN – La Cité de la Lumière** (Lyon Confluence) : **2 cellules jumelles**, **42 plaques lumineuses** pilotées chacune par **32 canaux** couvrant tout le spectre visible. Le logiciel de recherche étant trop complexe pour l'initiation, j'ai développé une **application web de serious games**, embarquée sur **Raspberry Pi 5**, **100 % hors-ligne**. Ma partie (**E2**) couvre la base de données, l'API, l'interface, les jeux solo et multijoueur, la mesure colorimétrique et la documentation.

2. Les faits à connaître par cœur

Matériel (chiffres officiels : slide commune de l'équipe)

- **2 cellules jumelles** (2 salles d'analyse identiques) · **42 plaques** au total (**21 par cellule**).
- **Une plaque** : **80 cm × 80 cm** (≈ 23 cm d'épaisseur) · **2360 LED** · ≈ **300 W** rayonnés au maximum · LED disposées **en bandes sur les bords**.
- **32 canaux de commande = 32 types de LED** : **24 couleurs à spectre étroit** (du **proche UV** au **proche IR**) + **8 blancs** à différentes températures (et IR).
- **Liaison matérielle** : **bus série RS-485** (différentiel, multi-points, robuste sur longue distance) → le logiciel de supervision adresse chaque plaque / chaque canal sur ce bus.
- **Pont logiciel** : [`supervision.exe`](#) (sur le Pi) reçoit les commandes de mon API et les traduit en trames **RS-485** vers les plaques.

⚠ **À vérifier avant l'oral** : ma fiche disait avant « 18 spectrales + 14 phosphore ». La **slide officielle de l'équipe** dit **24 spectre étroit + 8 blancs = 32**. Retenir **24 + 8** (chiffre de l'équipe). Total = 32 canaux dans les deux cas.

Accès / réseau (à connaître pour la démo)

- **URL d'accès** : <http://172.17.40.39/> : depuis une **tablette** ou **n'importe quel appareil** connecté au Wi-Fi local **ColorRoom_WiFi**. (En interne le conteneur expose 3000, publié sur 8080 ; sur le réseau de la salle l'app répond à cette adresse.)
- Colorimètre CS-150/160 en **USB** sur le Pi · **Portainer** sur le port **9000**.

Pile logicielle (la mienne, E2 / équipe JavaScript)

- Next.js **16.2.1** (App Router), React **19.2.4**, TypeScript **5.5.3** (strict), better-sqlite3 **11.5.0**, Three.js **0.160.1**, Docker (multi-stage **arm64**), Portainer, Raspberry Pi **5**.

Équipe (8 : 2 sous-équipes)

- **Équipe 1 : JavaScript / React (E1 → E4)** : E1 Bonnevey (infra/Docker/CI-CD), **E2 Trompier = moi** (BDD, API, UI, jeux solo+IA+multi, mesure, doc), E3 Arbadji Ilyes (éditeur no-code + planification), E4 Akyuz (tests API).
- **Équipe 2 : Python / NiceGUI (E5 → E8)**.
- **Partenaires** : LUMEN (Cité de la Lumière, Lyon Confluence) · ENTPE / LTDS : labo **BPMNP** (Bio-ingénierie, Perception, Mécanique Numérique) · contacts Labayrade & Vella · prof Delbosc.

3. Démonstration minutée (20 min)

Prépare une **vidéo de secours** de chaque étape (au cas où le matériel/Pi bug).

1. **Connexion / inscription** (2 min) : créer un apprenant, montrer la détection de rôle, la connexion auto.
2. **Catalogue + un jeu solo** (4 min) : lancer **Color Speed** : montrer les dalles s'allumer en temps réel (3D + vraies dalles), le score.
3. **Puissance 4 contre l'IA** (4 min) : choisir un niveau (Novice → Légendaire), montrer que l'IA bloque une menace (anti-piège), parler du minimax.
4. **Multijoueur** (3 min) : Spectre Chromatique : 2 terminaux rejoignent, projection d'une couleur, scores.
5. **Mesure CS-160** (3 min) : connecter, mesurer une dalle, lire X Y Z / Lv / x,y, point sur le diagramme CIE.
6. **Tableau de bord enseignant** (2 min) : classes, scores, export CSV.
7. **Coulisses** (2 min) : Portainer (conteneurs), le `docker compose`, la base SQLite (fichier).

4. Le code à connaître (fichiers clés)

- `lib/db/index.ts` : connexion SQLite `singleton` (`dbSingleton`), `migrate()` : `PRAGMA journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout=5000`, `foreign_keys=ON` ; tables `crg_*` créées en `CREATE TABLE IF NOT EXISTS` + `ALTER` protégés ; `seedAdminFromEnv()` (compte admin via `.env`).
- `lib/auth.ts` : `hashPassword` / `verifyPassword` (**PBKDF2-HMAC-SHA512**, 100 000 itérations, sel 16 o, clé 64 o, format `sel:hash`) ; `createSession` (token `randomBytes(32)`, **30 jours**), `renewSession` (fenêtre glissante).
- `app/api/auth/register/route.ts` : inscription en **transaction atomique** (`db.transaction`).
- `app/api/auth/login/route.ts` : `verifyPassword` + cookie `crg_session` **HttpOnly + SameSite=lax** (30 j).
- `app/api/supervision/batch/route.ts` : proxy LED : **sémaphore** `HW_CONCURRENCY=2` + file d'attente (supervision.exe est quasi-série) + `AbortController` (timeout).
- `app/_components/GamePuissance4.tsx` : IA **minimax** + **élagage alpha-bêta**, `scoreWindow` (heuristique fenêtres de 4), profondeurs **1/2/5/9/12**, anti-piège (`winningCols`).
- `app/_components/Room3D.tsx` : vue 3D Three.js (WebGLRenderer, `forceContextLoss` au démontage, pause via Page Visibility API).
- `lib/multiplayer.ts` + `app/api/multiplayer/state/route.ts` : sessions persistées en base (`crg_mp_sessions.state_json`), lues en **polling**.

5 extraits de code montrés dans le deck (fichier + lien GitHub + lignes) :

- **Three.js** (slide 20) : `Room3D.tsx` : Scene + WebGLRenderer + boucle de rendu + `forceContextLoss` .

- **Variable transactionnelle ACID** (slide 23) : `db.transaction()` de `register/route.ts` .

- **Variable atomique · sémaphore** (slide 24) : `hwInFlight` de `supervision/batch/route.ts` .

- **Variable volatile (useRef)** (slide 25) : `comboRef` / `useState` de `GameColorSpeed.tsx` .

- **Minimax (alpha-bêta)** (slide 34) : `scoreWindow` de `GamePuissance4.tsx` .

5. Fiche réponses du jury (Q&A)

Projet & cahier des charges

- « **À quoi sert la ColorRoom ?** » → Reproduire des éclairages à spectres précis pour la recherche sur la perception des couleurs ; mon appli la rend accessible en initiation via des jeux.
- « **Pourquoi des serious games ?** » → Le logiciel de recherche est trop technique ; le jeu rend la lumière/couleur compréhensible pour des apprenants.
- « **Qu'as-TU fait précisément ?** » → Partie E2 : BDD (7+ tables), API, UI/design, jeux solo + Puissance 4 (IA) + multijoueur, mesure CS-160, documentation. L'éditeur no-code est la partie E3.

Architecture & Next.js

- « **Pourquoi Next.js ?** » → Un seul projet front (React) + back (**Route Handlers**), un seul runtime Node à déployer sur le Pi ; **Server Components** pour le rendu, App Router.
- « **Architecture ?** » → Navigateur → routes API Next.js → couche `lib/` (auth, db, services) → SQLite + matériel (proxy supervision, pont CS-160). Tout conteneurisé.

Choix techniques (la question Node-RED)

- « **Pourquoi pas Node-RED comme proposé ?** » → Node-RED est **flow-based** : excellent pour automatiser des flux, mais inadapté à une vraie UI multi-pages (3D, catalogue, jeux), et ses flux JSON sont durs à versionner. React/TS/Next donnent des composants réutilisables, un **typage strict** (erreurs à la compilation), du code versionnable.
- « **Pourquoi TypeScript ?** » → Typage statique : un champ manquant ou une couleur mal formée est détecté par `tsc` /ESLint, pas en production.

Réseau / Docker / Raspberry Pi

- « **Comment déposes-tu ?** » → Image **Docker multi-stage** (deps → builder → runner) pour **arm64**, `docker compose up -d --build` , supervision par **Portainer**, app sur le port 8080.
- « **Et le réseau ?** » → Wi-Fi local **ColorRoom_WiFi** fourni par le Pi ; les tablettes/téléphones se connectent en local ; le colorimètre est en **USB** sur le Pi.
- « **Hors-ligne, vraiment ?** » → Oui : app + SQLite + matériel, tout local. (L'IA cloud Gemini est optionnelle ; un modèle local Ollama peut prendre le relais.)

Base de données / SQLite

- « **Pourquoi SQLite ?** » → Base **embarquée** (un simple fichier), zéro serveur à installer, idéale sur Pi ; via **better-sqlite3** (API **synchrone**, requêtes **préparées** = anti-injection).

- « **SQLite tient la charge ?** » → Pour quelques dizaines d'utilisateurs locaux, oui. **WAL** autorise des lectures concurrentes pendant une écriture, `busy_timeout` évite les erreurs `SQLITE_BUSY`.
- « **Migrations ?** » → `CREATE TABLE IF NOT EXISTS` + `ALTER` protégés : **idempotentes**, une base neuve se crée seule, une existante se met à jour sans casser.

Variables & persistance (questions « pointues »)

- **Atomique / transactionnelle** → `db.transaction()` (better-sqlite3) enveloppe `BEGIN/COMMIT/ROLLBACK` : l'inscription (user + adhésion classe) est **tout-ou-rien** (Atomicité ACID). Sinon : comptes « à moitié créés ».
- **Volatile (en mémoire)** → En JS il n'y a pas de mot-clé `volatile` ; je parle de l'**état runtime** stocké en `useRef` (ex. l'état des dalles pendant un jeu) : rapide, mais **non persistant**, perdu au rechargement : par opposition aux données **persistées** en SQLite.
- **Concurrente** → Le matériel (supervision.exe) est quasi-série ; un **sémaphore** `HW_CONCURRENCY=2` + file d'attente borne les appels simultanés.
- **Réactive** → `useState` : modifier la valeur déclenche le **re-rendu** ciblé de l'interface.
- **Environnement** → `process.env` (clé API, mot de passe admin) lue depuis `.env`, **hors Git**.
- **Persistante** → SQLite dans un **volume Docker** : survit aux redémarrages.

Sécurité / RGPD

- « **Comment stockes-tu les mots de passe ?** » → Jamais en clair : **PBKDF2-HMAC-SHA512**, 100 000 itérations, **sel aléatoire** 16 o, clé 64 o, format `sel:hash`. La vérification recalcule le hash et compare.
- « **Pourquoi PBKDF2 et pas bcrypt/argon2 ?** » → PBKDF2 est **natif** dans le module `crypto` de Node (aucune dépendance native à compiler sur arm64) ; 100 000 itérations donnent un coût correct. Argon2 serait plus moderne mais ajouterait une dépendance native.
- « **Les sessions ?** » → Token aléatoire (`randomBytes(32)`) en cookie **HttpOnly** (inaccessible au JS → anti-XSS) + **SameSite=lax** (anti-CSRF), **30 jours** avec **renouvellement glissant**.
- « **RGPD ?** » → Minimisation (un **pseudo** suffit, pas de nom/email) ; mots de passe hachés ; tout **local** (ENTPE) ; **droit à l'effacement** en cascade ; cloisonnement par rôle.

Jeux & temps réel

- « **Comment une dalle s'allume vite ?** » → Le navigateur ne parle pas directement au matériel : une **route proxy** relaie. Les **32 canaux** sont envoyés **en parallèle** (`Promise.all`, concurrence bornée) avec `timeout` (`AbortController`).
- « **La couleur à l'écran = la dalle ?** » → Oui, rendu **unifié** ; `CHANNEL_PROFILES` associe chaque canal à sa longueur d'onde réelle.

IA (Puissance 4)

- « **Explique ton IA.** » → **Minimax** : on simule l'arbre des coups, MAX pour l'IA / MIN pour l'adversaire ; **élagage alpha-bêta** coupe les branches inutiles ; **profondeur** limitée (1/2/5/9/12 selon le niveau). L'évaluation `scoreWindow` note chaque fenêtre de 4 cases : **défense pondérée plus fort que l'attaque** (-170 vs +130), victoire = `WIN_SCORE` (1 000 000), bonus de **centralité**. Anti-piège : l'IA ne donne pas une victoire immédiate à l'adversaire.
- « **Temps de réponse ?** » → Quasi instantané grâce à l'alpha-bêta et au poids central (exploration des bons coups en premier).

Multijoueur

- « **WebSockets ?** » → Non : l'état est **persisté en base** (`crg_mp_sessions.state_json`) et lu en **polling** de `/state`. Plus simple et robuste à déployer sur Pi, suffisant pour la cadence des jeux.

- « **Présence des joueurs ?** » → **Heartbeat** régulier ; jetons de session = **UUID** ; l'hôte (siège 1) démarre/arrête.

Mesure / colorimétrie

- « **Comment mesures-tu ?** » → Colorimètre **Konica Minolta CS-160** via un **pont .NET**. On lit le **tristimulus CIE XYZ**, la **luminance L_v (cd/m²)** et la **chromaticité (x, y)**, tracés sur le **diagramme CIE 1931**. Le CS-160 est **branché en USB** sur le Pi ; l'app l'appelle via une **API HTTP**.
- « **ΔE ?** » → Écart de couleur entre la mesure et la cible, utilisé pour scorer la précision dans les jeux de mesure.

Classes, QR code, multijoueur

- « **Comment un élève rejoint une classe ?** » → L'enseignant crée une classe et obtient un **code à 6 caractères** (ex. `CS5VHX`, sans 0/O ni 1/I) **et un QR code**. L'élève entre ce code à l'inscription (ou scanne le QR) → adhésion **atomique**.
- « **Et le code en multijoueur ?** » → Différent du code de classe : l'hôte d'une partie obtient un **code de salon** ; les autres le saisissent pour rejoindre la **session** (`crg_mp_sessions`), synchronisée en **polling**.

Méthode & gestion de projet

- « **Quelle méthode ?** » → **Agile** : développement **incrémental**, livraisons régulières, ajustements. **Échanges réguliers avec le client M. Labayrade** (directeur du labo **BPMNP**).
- « **Gestion du code ?** » → **Git/GitHub** : je développe sur la branche `ux-last` (intégration), puis je **merge** sur `main` (stable) ; ~280 commits clairs, **CI/CD** GitLab → build Docker → Pi.
- « **La documentation ?** » → **C'est moi qui l'ai rédigée** : guide technique, **15 diagrammes UML**, notice apprenant (l'aide), manuel d'installation (README).

Qualité / tests

- « **Tests ?** » → Vérification de **types (`tsc`)** + **build de production** à chaque étape ; tests de l'API et fiches de recette côté E4 ; transactions ACID, exports UTF-8 + BOM.

UML

- « **Différence ERD / diagramme de classes ?** » → L'**ERD** modélise les **tables relationnelles** (clés primaires/étrangères) ; le **diagramme de classes** modélise la **conception objet** (TypeScript).

6. Questions pièges : réponses courtes

- « **C'est toi qui as tout fait ?** » → Non, projet d'équipe ; je présente **ma partie E2** (le cœur de l'appli), qui s'appuie sur l'infra de E1 et est testée par E4.
- « **Et si une dalle tombe en panne ?** » → L'API gère par plaque, l'appli continue, le jeu ne plante pas.
- « **Combien de joueurs simultanés ?** » → 1 dalle/joueur (jusqu'à 42) en local, limité par le Wi-Fi de la salle.
- « **Pourquoi pas une vraie base serveur (PostgreSQL) ?** » → Surdimensionné pour un déploiement embarqué mono-poste hors-ligne ; SQLite suffit et simplifie l'install.
- « **L'IA générative dans le projet ?** » → Hors dossier E2 ; c'est une **exploration personnelle** (génération de jeux assistée par IA locale Ollama) que je peux montrer en bonus.

7. Script slide par slide : 42 slides (≈18 min parlé + vidéo 2 min)

Deck = `ColorRoom_Presentation.pptx` (42 slides, +1 vidéo). Le **texte mot-à-mot est aussi dans les NOTES du présentateur** de chaque slide PowerPoint (caché du public, visible en mode présentateur). 🗨️

= durée cible. ★ = ne jamais sacrifier · ⚡ = montrer vite si en retard.

Bloc A : contexte & projet (1 → 12) · ~4 min

#	Slide	🕒	À dire (résumé)
1	Titre	0:15	Qui je suis (E2), le projet, pour LUMEN/ENTPE.
2	Sommaire	0:15	Contexte → ma contribution → démo.
3	Contexte & partenaire	0:35	ENTPE/LTDS, équipe BPMNP, à LUMEN (Lyon Confluence).
4	La ColorRoom	0:40	2 cellules, plaques sur murs + plafond, 42 plaques.
5	La plaque	0:35	80×80 cm, 2360 LED, 32 canaux (24+8), RS-485.
6	Appli supervision actuelle	0:30	Logiciel chercheurs trop complexe → besoin d'un outil pédagogique.
7	Le besoin	0:25	Serious games, sur Pi, hors-ligne.
8	Cas d'utilisation	0:35	Apprenant (compte, classe, jouer, scènes, mesurer) ; enseignant hérite ; admin.
9	Équipe	0:25	8 étudiants, 2 sous-équipes (JS/React dont moi, Python).
10 ⚡	Gantt	0:20	Projet planifié, ma part (E2) du début à la fin.
11 ★	Gestion de projet	0:35	Agile + Kanban , client M. Labayrade , Git ux-last → main , docs par moi.
12	Transition E2	0:10	« Place à ma partie. »

Bloc B : architecture & choix (13 → 18) · ~3:15

#	Slide	🕒	À dire
13	Architecture	0:35	Route Handlers → lib → SQLite + matériel ; le Pi est sur le déploiement.
14 ⚡	Classes	0:25	Entités typées TypeScript.
15 ★	Choix techniques	0:55	Node-RED (relier des boîtes) inadapté ; React/TS/Next : blocs, erreurs avant lancement, un seul projet.
16	Pile technique	0:20	Next, React, TS, SQLite, Three.js, Docker.
17	Réseau & déploiement	0:40	Pi + Docker ; dalles RS-485, CS-160 USB, API ; Wi-Fi local.
18	Configuration	0:20	URL d'API à chaud, test /health, 32 canaux.

Bloc C : ma partie E2 (19 → 37) · ~8:30

#	Slide	⌚	À dire
19	Interface	0:30	3D temps réel, menu par rôle, couleur écran = couleur dalle.
20	Code Three.js	0:35	Scene + WebGLRenderer, un Mesh/dalle, boucle de rendu, nettoyage.
21	Base de données	0:30	SQLite, WAL , migrations auto, clés étrangères.
22	Typologie des variables	0:30	Survol : transactionnelle, atomique, volatile... (j'en montre 3).
23 ★	Code · transactionnelle (ACID)	0:40	<code>db.transaction()</code> user + classe, tout-ou-rien ; ACID = 4 garanties.
24 ★	Code · atomique (sémaphore)	0:40	<code>hwInFlight</code> borné à 2, file ; mono-thread .
25 ★	Code · volatile (useRef)	0:35	<code>useRef</code> mémoire sans re-rendu, perdu au reload ; on persiste juste le score.
26	Sécurité	0:30	Mots de passe hachés , cookie sécurisé, tout local (RGPD).
27 ⚡	Séquence connexion	0:18	Vérif mot de passe → session 30 j → cookie.
28	Tableau de bord enseignant	0:30	Classe = code + QR , suivi élèves, export CSV.
29	Parcours apprenant	0:25	Inscription 3 étapes + code de classe, transaction, connexion auto.
30	Jeux (du clic aux dalles)	0:30	Mes jeux (Color Speed, Simon, Tetris, P4, mesure) ; clic → moteur → dalles → score.
31 ⚡	Remappage des canaux	0:15	Couleur RGB → 32 canaux réels.
32 ⚡	États d'une partie	0:12	Prête → en cours → terminée.
33 ★	IA Puissance 4	0:50	Minimax + alpha-bêta , 5 niveaux (profondeur), défense > attaque. Pas un réseau de neurones.
34 ★	Code minimax	0:35	Fenêtre de 4 + coupure alpha-bêta.
35	Jeux multijoueur	0:30	Sans WebSocket : état en base lu en polling ; code de salon .
36 ⚡	Séquence multijoueur	0:18	Hôte crée → invité saisit le code → synchro.
37	Éditeur + IA	0:30	Éditeur no-code = E3 ; mon exploration : « Créer avec l'IA » (Ollama local).

Bloc D : clôture + vidéo (38 → 42) · ~1:30 (+2 min vidéo)

#	Slide	🕒	À dire
38	Aide & qualité	0:30	Aide /aide, doc + 15 UML par moi ; types/build, tests API par E4.
39	Difficultés	0:30	CORS & pare-feu (→ suivante), latence, accès concurrents, multi.
40	Notes réseau Windows	0:20	CORS : ouverture du port + portproxy 18080 → 8080.
41	Place à la démonstration	0:10	« On passe à la démonstration sur le matériel réel. »
42 ★	Vidéo du projet	2:00	Lancer la vidéo (ColorRoom réelle) → enchaîner sur la démo.

Total ≈ 18 min parlé + 2 min vidéo = 20:00. En retard → couper les ⚡. Ne jamais sacrifier les ★ (11, 15, 23, 24, 25, 33, 34, 42).

💡 Le **texte mot-à-mot complet** est dans les **notes du présentateur** du `.pptx` : en mode diaporama, active l'**affichage présentateur** pour le voir sur ton écran sans que le public le voie.

8. Mémo « infos complexes à ne pas oublier »

- **ColorRoom** : 2 cellules ; plaques **murs + plafond** ; **42 plaques** (21/cellule).
- **Plaque** : 80×80 cm, 2360 LED, ~300 W, **32 canaux = 24 spectres étroits + 8 blancs, RS-485**.
- **Sigles** : ENTPE (École Nat. des Travaux Publics de l'État), LTDS (Lab. de Tribologie et Dynamique des Systèmes), BPMNP (Bio-ingénierie, Perception, Mécanique Numérique), LUMEN (Cité de la Lumière).
- **ACID** = Atomicité, Cohérence, Isolation, Durabilité.
- **Méthode** : agile + Kanban ; client **M. Labayrade** ; Git **ux-last** → **main** ; docs par moi.
- **Code de classe** (+ QR) ≠ **code de salon** multijoueur.
- **CS-160** : USB sur le Pi, appelé via API. **WAL** : lectures concurrentes. **Minimax** : pas un réseau de neurones.
- **URL démo** : <http://172.17.40.39/> (Wi-Fi ColorRoom_WiFi).

9. La vidéo : slide 42 (embarquée dans le .pptx)

Slide-pont présentation → démo. Je la lance à la fin du parlé, puis j'enchaîne sur la démonstration live. C'est le seul moyen de montrer la vraie ColorRoom (à LUMEN), et un filet de sécurité si le matériel bug.

Bon courage Téo : section 5 = 90 % des questions, section 7 = ton minutage (42 slides, 20 min), et le mot-à-mot est dans les notes du PowerPoint.